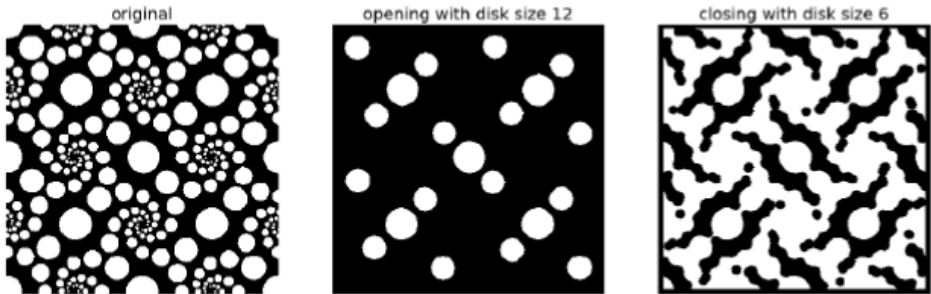


영상 처리 사례 (Image Processing Use-Case)

1) 설명

6주차까지 진행된 파이썬 코드 중 가장 관심과 흥미를 갖게 된 코드를 **2개 이상** 선택하고 그 활용 결과를 정리합니다. 선택한 코드에 관심 분야(예, 교통, 의료 등)의 데이터(2차원 영상)를 입력으로 하여 그 결과를 분석하는 **영상 처리 사례 보고서**를 작성합니다.

2) 예시

관심 코드 1	morphop.ipynb
코드 내용	1 2차원 모폴로지 연산 수행 1 침식과 팽창 또는 열림과 닫힘 기능 구현
입력 데이터	원의 불규칙 영상
수행 결과	

3) 요구사항

수업 시간에 소개된 다양한 파이썬 코드를 수정하여 활용한다.

코드 중 가장 관심과 흥미 대상이 되었던 코드를 선택한다.

관심 분야(예, 교통 의료 등)의 데이터를 검색하여 영상 파일로 활용한다.

그 영상 파일을 수정된 파이썬 코드의 입력으로 하여 결과를 도출하고 분석한다.

4) 평가 기준

관심 코드의 내용 이해도 (30%)

관심 분야의 입력 데이터 적정성 (40%)

분석 및 서술의 완성도 (30%)

영상 처리 사례 보고서

주제명	산불 감지를 위한 영상처리 기법 구현 및 시각화	
작성자	성명	E-mail
	권영훈	20213032@office.deu.ac.kr

< 코드 내용 정리 >

관심 코드	chromakeying.ipynb morphop.ipynb
코드 내용	■ chromakeying.ipynb: - HSV 색공간 변환: 빨간색 추출을 위한 HSV 변환. - 빨간색 범위 마스크: 불꽃 영역 추출. - 다중 스펙트럼 영상 합성: 원본 이미지와 불꽃, 타버린 영역 합성 이미지 추출. ■ morphop.ipynb: - 모폴로지 연산: 오픈닝, 팽창 연산을 사용하여 노이즈 제거 및 불꽃 확장.

< 입력 데이터 >

입력 데이터: 영상 파일	참고 링크
	“전 시민 대피하라” 도로에서도 보이는 안동 산불 상황(영상) https://v.daum.net/v/20250325183121800

〈 수행 결과 〉

처리 과정 결과 이미지	Original Image Fire & Burnt Overlay Red: Fire, Blue: Burnt Area 
주제 선정 이유	<주제 선정 이유> 2025 년 4 월 5 일, 조모님의 장례식 3 일차에 화장 절차를 마친 후, 유골함을 경북 안동에 있는 생가 근처 선산에 안장하러 가는 길이었다. 선산에는 산불로 인해 타버린 조부님의 묘가 있었고, 그곳에 함께 합장하고자 버스를 타고 이동 중이었다. '동안동 CC' 인근을 지나던 중, 최근 발생한 산불로 산 전체가 검은 재로 변해버린 광경을 직접 목격하게 되었고, 그 장면은 큰 충격으로 다가왔다. 검게 그을린 산봉우리와 마을의 경관을 바라보며, 한때 평화로웠던 이 장소가 순식간에 잿더미로 변했다는 사실을 믿기 어려웠다. 화재 피해 현장을 직접 보기 전까지는 피해가 이토록 심각할 것이라 생각하지 못했기에, 단순히 사진을 넘어서 기술적으로 산불이 지나간 흔적과 피해 지역을 시각적으로 보여줄 수 있다면, 많은 이들에게 경각심을 줄 수 있을 것이라는 생각이 들었다. 이러한 개인적인 경험은 이번 과제의 주제를 '산불 감지'로 정하게 된 결정적인 계기가 되었으며, 영상처리 기술을 통해 자연재해의 심각성을 인식하고 효과적으로 대응하는 데 기여하고자 하는 의지를 더욱 확고히 다져주었다.

<프로젝트 개요>

본 과제는 산불이 발생한 지역의 이미지를 입력으로 받아, 불꽃이 타오르는 영역과 이미 타버린 지역을 자동으로 감지하고 시각화 하는 영상처리 시스템을 구현하는 것이다. OpenCV 와 matplotlib 를 활용하였으며, 주요 기법으로는 색상 기반 마스킹(Hue-Saturation-Value 변환), 모폴로지 연산(Morphological Operation), 마스킹 및 영역 합성 기술 등이 사용되었다.

<주요 처리 과정 요약>

1. 불꽃 감지 (Fire Detection)
2. 타버린 영역 감지 (Burnt Area Detection)
3. 영역 시각화 (Overlaying Detected Regions)

<사용한 주요 코드 설명>

산불 화재 이미지에서 불꽃 및 타버린 영역을 자동으로 감지하여 HSV 색상 범위로 추출하고, 모폴로지 연산으로 노이즈를 제거하고 확장하며 마스킹을 적용한 후 시각화 합니다.

다음은 주요 처리 과정에 대한 설명입니다.

처리

과정

분석

1. HUV 색상 공간으로 변환

- `hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)`
 - HSV 색상 공간은 특정 색상(예: 빨간색, 어두운색)을 보다 쉽게 추출할 수 있습니다.
 - HSV 는 색조(Hue), 채도(Saturation), 명도(Value)로 구성되어 있으며, 각각 색상/선명도/밝기를 의미합니다.
 - 이 코드는 불꽃(빨간색 계열)과 타버린 영역(어두운 색)을 찾기 위해 HSV 변환을 먼저 수행합니다.

2. 불꽃 영역 추출

- `mask1 = cv2.inRange(hsv, lower_red1, upper_red1)`
`mask2 = cv2.inRange(hsv, lower_red2, upper_red2)`
`red_mask = cv2.bitwise_or(mask1, mask2)`
 - 불꽃은 강한 빨간색이기 때문에, HSV 색공간에서 빨간색 범위(Hue)를 지정하여 마스킹합니다.
 - HSV 에서 빨간색은 Hue 값이 010과 160180 범위에 양쪽 끝으로 분포하므로, 두 개의 범위를 나누어 각각 마스크를 생성합니다.
 - 이후 `cv2.bitwise_or()` 함수를 사용해 두 마스크를 병합함으로써, 전체 빨간색 영역을 포함하는 하나의 통합 마스크 `red_mask` 를 만듭니다.
 - 최종적으로 `red_mask` 는 이미지에서 불꽃이 있는 영역은 흰색(255)으로, 그 외 영역은 검은색(0)으로 이진화 된 형태로 표현됩니다.

3. 노이즈 제거 (형태학적 연산: Opening)

- `red_mask_clean = cv2.morphologyEx(red_mask, cv2.MORPH_OPEN, kernel)`
 - 마스크에는 종종 작은 점이나 불필요한 픽셀이 섞여 있습니다. 이를 노이즈라고 합니다.
 - MORPH_OPEN 은 침식 → 팽창 순서로 수행되어 작은 잡음을 제거하면서 객체의 형태는 유지합니다.
 - 결과적으로 더 선명하고 명확한 불꽃 영역만 남게 됩니다.

4. 타버린 영역 감지

1. 불꽃 주변 확대

- `red_mask_dilated = cv2.dilate(red_mask_clean, kernel_dilate, iterations=2)`
 - 불꽃이 있던 위치 근처를 조사하기 위해 해당 마스크를 팽창시킵니다.
 - 팽창(Dilation)은 하얀 영역(불꽃)을 주변으로 확장시켜, "불꽃 근처"를 포함하는 영역을 생성합니다.

2. 어두운 영역 마스크 생성

- `burnt_candidate_mask = cv2.inRange(hsv, lower_burnt, upper_burnt)`
 - 타버린 영역은 일반적으로 어둡고 채도가 낮기 때문에, HSV 기준으로 어두운 색상 범위를 지정합니다.
 - 이 범위에 해당하는 픽셀만 추출하여 "타버린 지역 후보 영역"을 생성합니다.

3. 불꽃 근처 어두운 영역만 추출

- `burnt_mask_near_fire = cv2.bitwise_and(burnt_candidate_mask, red_mask_dilated)`
 - 모든 어두운 영역이 타버린 지역은 아닐 수 있기 때문에, 불꽃 주변에 위치한 어두운 영역만 골라냅니다.
 - 이렇게 하면 산불에 의해 실제로 영향을 받은 부분만 `burnt_mask_near_fire`로 남게 됩니다.

5. 시각화용 마스크 이미지 생성

```
■ overlay = np.zeros_like(image_rgb)
overlay[red_mask_clean > 0] = [255, 0, 0]
# 불꽃: 빨간색
overlay[burnt_mask_near_fire > 0] = [0, 0, 255]
# 타버린 지역: 파란색
```

- 시각화를 위해 결과 이미지를 구성합니다.
- 불꽃 영역은 **빨간색 (RGB: 255,0,0)**, 타버린 영역은 **파란색 (0,0,255)**으로 색상을 입힙니다.
- 이 오버레이 이미지는 나중에 원본 이미지 위에 반투명하게 합쳐져서 출력됩니다.

6. 이미지 합성 및 결과 출력

```
■ overlaid_image = cv2.addWeighted(image_rgb, 0.7,
overlay, 0.3, 0)
```

- 원본 이미지와 불꽃/타버린 마스크를 **70 : 30 비율**로 합쳐서 직관적인 시각화 이미지를 생성합니다.
- 이 이미지와 함께 **원본 이미지, 불꽃 + 타버린 영역 마스크**를 나란히 비교하여 확인할 수 있도록 **matplotlib**으로 3 가지 이미지를 보여줍니다.

7. 처리 과정 요약 표

단계	처리 내용	사용 기술
1	HSV 색공간 변환	cv2.cvtColor
2	불꽃 마스크	cv2.inRange, bitwise_or
3	노이즈 제거	cv2.morphologyEx
4	타버린 지역 감지	cv2.dilate, bitwise_and
5	시각화 처리	cv2.addWeighted, matplotlib

8. 요약

- 불꽃이 발생한 영역은 일반적으로 강한 빨간색 계열의 색상을 띠고, 이미 타버린 지역은 낮은 밝기와 채도를 가지는 어두운 색상으로 나타납니다. 이러한 색상의 특징을 기반으로, 영상의 HSV 색공간에서 각각 빨간색과 어두운 색상에 해당하는 영역을 마스크로 추출합니다.
- 추출된 마스크에는 잡음이나 작은 불필요한 객체가 포함될 수 있으므로, 형태학적 연산(Morphological Operation)을 통해 해당 영역들을 정제합니다.
- 구체적으로는 침식(Erosion)과 팽창(Dilation)을 조합한 Open 연산을 통해 작은 점 잡음을 제거하고, 불꽃 주변 영역을 확장하기 위해 팽창(Dilation) 연산을 추가로 적용합니다.
- 이후에는 불꽃이 감지된 주변의 어두운 영역만을 필터링하는 위치 기반 연산을 통해, 실제 화재로 인해 손상되었을 가능성이 높은 지역만을 최종적으로 분리해냅니다.
- 최종적으로, 감지된 불꽃 영역은 빨간색, 타버린 영역은 파란색으로 시각화 하여, 사람이 직관적으로 화재 진행 상황과 피해 범위를 한눈에 파악할 수 있도록 결과 이미지를 구성합니다.

9. Google Colab Link

- https://colab.research.google.com/drive/19at7cFfvr-7J1P7M_-MYeTzl1rxfy2HU?usp=sharing

10.소스코드

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread('/content/drive/MyDrive/task/안동산불.jpeg')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```



```

hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# 불꽃 영역 감지 (HSV 기준)
lower_red1 = np.array([0, 100, 100])
upper_red1 = np.array([10, 255, 255])
lower_red2 = np.array([160, 100, 100])
upper_red2 = np.array([180, 255, 255])

mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
red_mask = cv2.bitwise_or(mask1, mask2)

# 노이즈 제거 (morphology)
kernel = np.ones((5, 5), np.uint8)
red_mask_clean = cv2.morphologyEx(red_mask, cv2.MORPH_OPEN,
kernel)

# 타버린 지역 감지: 불꽃 주변 + 어두운 영역
# 1. 불꽃 주변 dilate (주변 영역 확장)
kernel_dilate = np.ones((7, 7), np.uint8)
red_mask_dilated = cv2.dilate(red_mask_clean, kernel_dilate,
iterations=2)

# 2. 어두운 영역 마스크 만들기 (저채도 + 저명도)
lower_burnt = np.array([0, 0, 0])
upper_burnt = np.array([180, 100, 120])
burnt_candidate_mask = cv2.inRange(hsv, lower_burnt,
upper_burnt)

# 3. 불꽃 주변에 있는 어두운 영역만 추출
burnt_mask_near_fire = cv2.bitwise_and(burnt_candidate_mask,
red_mask_dilated)

# 빨간색 영역과 파란색 영역 시각화용 색상 임하기
overlay = np.zeros_like(image_rgb)
overlay[red_mask_clean > 0] = [255, 0, 0] # Red: 불꽃
overlay[burnt_mask_near_fire > 0] = [0, 0, 255] # Blue: 타버린
지역

# 이미지 합성
overlayed_image = cv2.addWeighted(image_rgb, 0.7, overlay,
0.3, 0)

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis('off')

```

```
plt.subplot(1, 3, 2)
plt.imshow(overlayed_image)
plt.title("Fire & Burnt Overlay")
plt.axis('off')

plt.subplot(1, 3, 3)
combined_mask = np.zeros_like(image_rgb)
combined_mask[red_mask_clean > 0] = [255, 0, 0]
combined_mask[burnt_mask_near_fire > 0] = [0, 0, 255]
plt.imshow(combined_mask)
plt.title("Red: Fire, Blue: Burnt Area")
plt.axis('off')

plt.tight_layout()
plt.show()
```

<참고 문헌: URL, 도서 등>

1. 대표 자료

- <https://opencv-python.readthedocs.io/en/latest/>

2. HSV 색공간에서 이미지의 특정 색 추출.

- https://velog.io/@nayeon_p00/OpenCV-Python-HSV-%EC%83%89%EA%B3%B5%EA%B0%84%EC%97%90%EC%84%9C-%EC%9D%B4%EB%AF%B8%EC%A7%80%EC%9D%98-%ED%8A%B9%EC%A0%95%EC%83%89-%EC%B6%94%EC%B6%9C%ED%95%98%EA%B8%B0

3. opencv 입문하기 7-2편 HSV, 특정 색상 추출(inRange)

- <https://sikaleo.tistory.com/84> [SIKALEO:티스토리]

4. [OpenCV] 2. Mask 이미지, 이미지 합성

- <https://computistics.tistory.com/34>

5. [OpenCV] 이미지 ROI 마스크 적용 방법들, 컬러 마스크 중복 적용하기

- <https://ai-crumble.tistory.com/entry/OpenCV-다양한-방법으로-이미지-마스크-적용-컬러-마스크-중복-적용-하기-bitwise-copyTo-addWeighted#05> [조각 지식 모으기:티스토리]

6. OpenCV - 9. 이미지 연산 (합성, 알파 블렌딩, 마스크)

- <https://bkshin.tistory.com/entry/OpenCV-9-%EC%9D%B4%EB%AF%B8%EC%A7%80-%EC%97%B0%EC%82%B0>

7. 6. OpenCV C++ 이미지 침식, 팽창 (erode, dilate) 를 통해 노이즈 제거, 없는 부분 채우기

- [6. OpenCV C++ 이미지 침식, 팽창 \(erode, dilate\) 를 통해 노이즈 제거, 없는 부분 채우기](#)|작성자 [김진혁](#)